

Assignment Two

EXTENDED

File Handling in SV

By: Eng. Islam Hamdy Mohamed

Guided By: Eng. Hassan Khaled

1. Snippets for the implemented code

```
// Using 'logic' instead of 'reg' or 'wire' because it is a 4-valued logic data type
// (0, 1, X, Z) that permits both procedural assignments and continuous assignments,
// preventing any net vs. variable type conflicts in SystemVerilog.
typedef struct packed {
    logic [7:0] data_in;
    bit  [1:0] op_sel;
} stim_t;

module multi_op_processor_tb;

    logic [7:0] data_in;
    bit  [1:0] op_sel;
    logic [7:0] data_out_dut;

    stim_t      stimulus_array[];
    logic [7:0] Queue_Output[$];
    logic [7:0] Expected_Values[int];

    // Reads binary tokens from a file, checks bit width, and pushes them into a queue
    task automatic read_bin_file (
        input string file_name,
        input int width,
        ref logic [31:0] q[$]
    );
        int file_desc;
        logic [31:0] temp_val;

        q.delete(); // Clear queue before reading

        file_desc = $fopen(file_name, "r");
        if (file_desc == 0) begin
            $fatal(1, "[FATAL] Failed to open file: %s", file_name);
        end

        // Read binary values until end-of-file
        while (!$feof(file_desc)) begin
            if ($fscanf(file_desc, "%b\n", temp_val) == 1) begin
                // Terminate if any value exceeds the expected bit width
                if (temp_val >= (1 << width)) begin
                    $fatal(1, "[FATAL] Width mismatch in file %s. Value %b exceeds %0d bits.",
                        file_name, temp_val, width);
                end
                q.push_back(temp_val);
            end
        end

        $fclose(file_desc);
    endtask
```

Figure 1 Declaration of Functions and Tasks {1}

```

// Loads stimulus and golden files, unpacks input tokens, and fills arrays
task automatic load_file_stimulus();
    logic [31:0] temp_stim_q[$];
    logic [31:0] temp_gold_q[$];

    // Read files into temporary queues
    read_bin_file("inputs.txt", 10, temp_stim_q);
    read_bin_file("expected_Out.txt", 8, temp_gold_q);

    stimulus_array = new[temp_stim_q.size()];

    // Unpack 10-bit token: upper 8 bits to data_in, lower 2 bits to op_sel
    foreach (temp_stim_q[i]) begin
        stimulus_array[i].data_in = temp_stim_q[i][9:2];
        stimulus_array[i].op_sel = temp_stim_q[i][1:0];
    end

    // Store golden results in the associative array
    foreach (temp_gold_q[i]) begin
        Expected_Values[i] = temp_gold_q[i][7:0];
    end

    $display("[INFO] Successfully loaded %0d stimulus vectors and %0d expected golden values.",
        stimulus_array.size(), Expected_Values.num());
endtask

task drive_stim_and_collect ();
    foreach(stimulus_array[i]) begin
        data_in = stimulus_array[i].data_in;
        op_sel = stimulus_array[i].op_sel;
        #1ns;
        collect_output_data();
    end
endtask

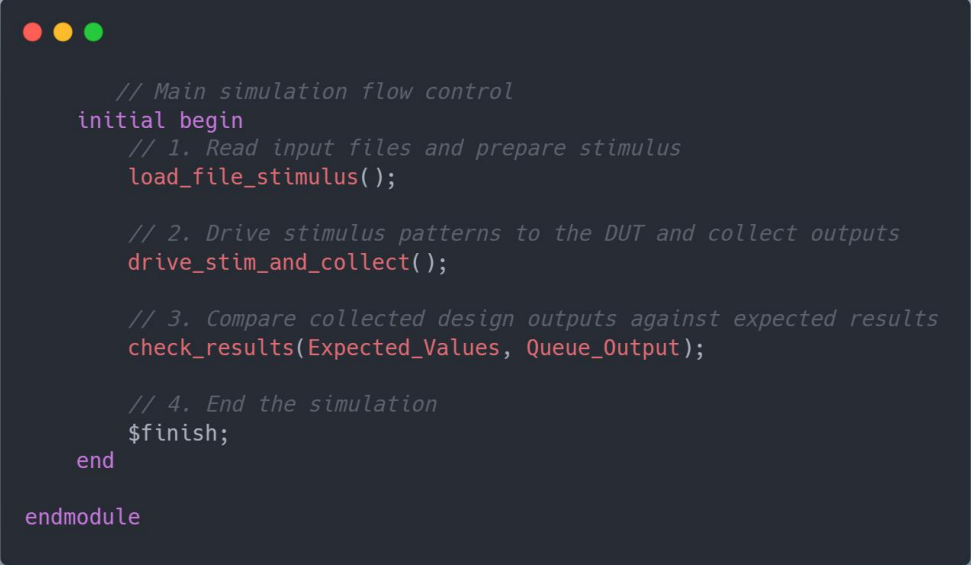
function void collect_output_data ();
    Queue_Output.push_back(data_out_dut);
endfunction

task check_results (logic [7:0] Expected_Values[int], logic [7:0] Queue_Output[$]);
    foreach(Expected_Values[i]) begin
        if (Expected_Values[i] != Queue_Output[i]) begin
            $display("Mismatch at index %0d: Expected %0d, Got %0d", i, Expected_Values[i],
                Queue_Output[i]);
        end else begin
            $display("Match at index %0d: Value %0d", i, Expected_Values[i]);
        end
    end
endtask

multi_op_processor DUT(
    .data_in(data_in),
    .op_sel(op_sel),
    .data_out(data_out_dut)
);

```

Figure 2 Figure 1 Declaration of Functions and Tasks {2}



```

// Main simulation flow control
initial begin
    // 1. Read input files and prepare stimulus
    load_file_stimulus();

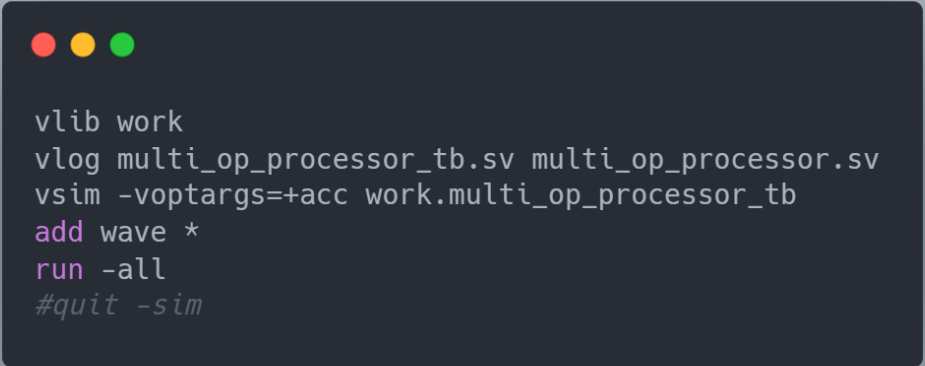
    // 2. Drive stimulus patterns to the DUT and collect outputs
    drive_stim_and_collect();

    // 3. Compare collected design outputs against expected results
    check_results(Expected_Values, Queue_Output);

    // 4. End the simulation
    $finish;
end
endmodule

```

Figure 3 Initial Block (Main Code)



```

vlib work
vlog multi_op_processor_tb.sv multi_op_processor.sv
vsim -voptargs=+acc work.multi_op_processor_tb
add wave *
run -all
#quit -sim

```

Figure 4 Do file code for automating the process.

2. Snippet for the waveform shows the values for created variables



Figure 5 waveform shows the values for created variables

3. Snippets for the logs show the all printed values.

```
# [INFO] Successfully loaded 200 stimulus vectors and 200 expected golden values.
# Match at index 0: Value 29
# Match at index 1: Value 212
# Match at index 2: Value 169
# Match at index 3: Value 127
# Match at index 4: Value 17
# Match at index 5: Value 184
# Match at index 6: Value 124
# Match at index 7: Value 138
# Match at index 8: Value 241
# Match at index 9: Value 17
# Match at index 10: Value 132
# Match at index 11: Value 83
# Match at index 12: Value 119
# Match at index 13: Value 11
# Match at index 14: Value 165
# Match at index 15: Value 186
# Match at index 16: Value 186
# Match at index 17: Value 43
# Match at index 18: Value 74
# Match at index 19: Value 227
# Match at index 20: Value 152
# Match at index 21: Value 254
# Match at index 22: Value 28
# Match at index 23: Value 106
# Match at index 24: Value 19
# Match at index 25: Value 210
# Match at index 26: Value 142
# Match at index 27: Value 170
# Match at index 28: Value 10
# Match at index 29: Value 54
# Match at index 30: Value 158
# Match at index 31: Value 5
# Match at index 32: Value 217
# Match at index 33: Value 3
# Match at index 34: Value 55
# Match at index 35: Value 230
# Match at index 36: Value 117
```

Figure 6 LOG {1}

```
# Match at index 37: Value 186
# Match at index 38: Value 124
# Match at index 39: Value 60
# Match at index 40: Value 17
# Match at index 41: Value 70
# Match at index 42: Value 87
# Match at index 43: Value 13
# Match at index 44: Value 59
# Match at index 45: Value 13
# Match at index 46: Value 245
# Match at index 47: Value 129
# Match at index 48: Value 79
# Match at index 49: Value 0
# Match at index 50: Value 21
# Match at index 51: Value 78
# Match at index 52: Value 253
# Match at index 53: Value 198
# Match at index 54: Value 88
# Match at index 55: Value 240
# Match at index 56: Value 230
# Match at index 57: Value 14
# Match at index 58: Value 156
# Match at index 59: Value 116
# Match at index 60: Value 162
# Match at index 61: Value 24
# Match at index 62: Value 130
# Match at index 63: Value 6
# Match at index 64: Value 182
# Match at index 65: Value 123
# Match at index 66: Value 215
# Match at index 67: Value 72
# Match at index 68: Value 130
# Match at index 69: Value 66
# Match at index 70: Value 88
# Match at index 71: Value 231
# Match at index 72: Value 15
# Match at index 73: Value 238
# Match at index 74: Value 218
```

Figure 7 LOG {2}